Rapport de TP1 — Simulation et Exploitation de Cache Mémoire

CSC 374 — Programmation Systèmes II

Établissement : iPNet **Filière :** Génie Logiciel (3ème année) **Binôme :** GNOUNKABITI Charôn & ALI Pouwèdéou

Partie A — Conception et Réalisation du Simulateur de Cache (csim.c)

1. Représentation du cache et structures de données

La hiérarchie de structures repose sur trois types imbriqués : LigneCache (valid, tag, timestamp), EnsembleCache (tableau de LigneCache*), et Cache (tableau d'EnsembleCache* + paramètres S, E, b). L'allocation dynamique est pilotée par les paramètres s, E et b passés en argument : S = 2^s ensembles de E lignes chacun. Aucune taille n'est codée en dur, garantissant la généricité complète du simulateur.

2. Décomposition des adresses 64 bits

Chaque adresse 64 bits est décomposée via arithmétique non signée (uint64_t) pour éviter tout piège d'extension de signe. Les trois champs sont extraits par masquage et décalage :

- Décalage dans le bloc (b bits) : offset = addr & ((1ULL << b) - 1)
- Indice d'ensemble (s bits) : set_index = (addr >> b) & ((1ULL << s) - 1)
- Étiquette (64 − s − b bits) : tag = addr >> (s + b)

Le type uint64_t est imposé sur toutes les opérations pour garantir la correction sur architectures 32 et 64 bits.

3. Algorithme LRU exact

Un compteur global (access_counter) est incrémenté à chaque accès mémoire et affecté au champ timestamp de la ligne touchée. En cas d'éviction sur ensemble plein, le simulateur parcourt les E lignes et sélectionne celle dont le timestamp est minimal, reproduisant fidèlement la politique Least Recently Used. Cette approche évite toute structure de liste chaînée et offre un comportement exact en O(E) par accès.

4. Robustesse et gestion des erreurs

Les arguments sont validés via getopt avec contrôle des bornes (E ≥ 1, s+b < 64). Les fichiers de trace sont ouverts avec vérification de retour ; les lignes malformées ou de type 'I' sont ignorées. La libération mémoire (free_cache) est systématique : chaque malloc est apparié à un free, ce que prouve le rapport Valgrind :

- All heap blocks were freed — no leaks are possible
- 23 allocs, 23 frees, 11 088 octets alloués — 0 erreur mémoire, 0 fuite

5. Résultats — 8/8 points (tous cas validés)

Trace	s / E / b	Hits	Misses	Évictions	Score
exercice2.trace	1 / 2 / 2	4	5	2	✓
entr4.trace	2 / 1 / 3	4	6	2	✓
cyclique_13.trace	6 / 12 / 6	0	13 000	12 988	✓
prodscal_0.trace	6 / 1 / 6	0	4 096	4 032	✓
ordre_colonnes_64.trace	4 / 4 / 6	3 840	4 354	4 290	✓
trans_32x32.trace	0 / 32 / 5	896	1 152	1 120	✓
histo.trace	4 / 2 / 4	5 660	521	489	✓
histo.trace	8 / 4 / 6	6 050	131	0	✓

Partie B — Optimisation de la Transposition de Matrice (trans.c)

1. Modèle du cache cible et analyse des conflits

Le cache cible est un cache direct-mapped (E=1) de 32 ensembles (s=5) avec des blocs de 32 octets (b=5), soit une capacité totale de 1 Ko. Chaque bloc stocke 8 entiers (int = 4 octets). Les matrices A et B de taille N×N sont séparées en mémoire par un décalage de 256 Ko, qui est un multiple exact de 1 Ko. Sur la diagonale, A[i][i] et B[i][i] tombent donc dans le même ensemble de cache, provoquant un conflit systématique et coûteux lors de la transposition naïve.

2. Cas 32×32 — Blocking 8×8 avec variables registres

La technique de tuilage (blocking) en sous-blocs 8×8 réduit les conflits de mapping. Le conflit diagonal — inévitable car A[i][i] et B[i][i] partagent le même ensemble — est résolu par une lecture complète des 8 entiers d'une ligne de A dans 8 variables locales (candidates à la promotion en registres), suivie de l'écriture dans B. Cette séquence évite l'éviction prématurée du bloc A actif. Résultat : 284 défauts contre 1 180 en naïf (seuil max : 300).

3. Cas 64×64 — Partition 4×4 avec zone tampon

Pour une matrice 64×64, le pas de colonne est de 64 × 4 = 256 octets = 8 ensembles. Les blocs de cache entrent donc en conflit tous les 4 rangs de la matrice, rendant le blocking 8×8 insuffisant. La stratégie retenue partitionne la matrice en sous-blocs 4×4 et exploite le quadrant haut-droit de B comme zone de stockage temporaire pour les éléments de la diagonale, évitant d'évincer les blocs actifs de A et B. Résultat : 1 224 défauts contre 4 720 en naïf (seuil max : 1 300).

4. Cas 61×67 — Blocking 16×16 avec gestion des bordures

Les dimensions non multiples de puissances de 2 réduisent naturellement les phénomènes de résonance de cache (les conflits ne se répètent pas périodiquement). Un bloc 16×16 exploite cette irrégularité spatiale pour maximiser la réutilisation des lignes de cache chargées. Les bordures (lignes/colonnes hors-multiple) sont traitées par un test conditionnel (if i < M && j < N) garantissant la correction sans biais de performance majeur. Résultat : 1 989 défauts contre 4 420 en naïf (seuil max : 2 000).

5. Résultats — 9/9 points (tous seuils franchis)

Matrice	Naïf	Optimisé	Seuil max	Gain	Score
32×32	1 180	284	300	−76%	3.0 / 3
64×64	4 720	1 224	1 300	−74%	3.0 / 3
61×67	4 420	1 989	2 000	−55%	3.0 / 3

Score total : 17 / 17 points (Partie A : 8/8 + Partie B : 9/9)